

UsefulBI DevOps Interview Q&A Master Guide

Version 1.1 FIXED - Questions, Answers, Workflows, Cheat Sheets, Memory Tricks, and System Design Architecture

Prepared from a senior DevOps / Platform / Cloud interviewer perspective for FAANG-style, Cisco, Meta, Palo Alto, Oracle, Zscaler, Zoom, AppZen, UsefulBI, and Bay Area startup interviews. This version uses fully wrapping text blocks and tables to prevent page clipping.

0. How to Use This PDF

Use this guide in three passes: first memorize the cheat sheets and workflows, then practice the questions topicwise, then answer the system design prompts out loud using the architecture diagrams. For each answer, interviewers usually grade: correctness, production experience, tradeoffs, troubleshooting steps, and security awareness.

Memory trick / clue: Answer every scenario using the same loop: Scope -> Observe -> Isolate -> Fix -> Prevent. This makes answers sound senior and structured.

1. UsefulBI-Focused Skills Matrix

Area	What interviewers expect	Best answer signals
AWS Cloud	VPC, IAM, EC2, EKS, S3, Glue, Redshift, Lambda, ALB/NLB, CloudWatch, cost controls	Can design secure, scalable cloud/data platforms; understands private networking, IAM least privilege, monitoring, and cost tradeoffs.
Terraform / IaC	Remote state, modules, drift, imports, plan safety, workspaces, lifecycle protections	Can run infrastructure safely through PR review, state locking, plan approvals, and environment separation.
Kubernetes / EKS	Pods, Deployments, Services, Ingress, probes, autoscaling, IRSA, upgrades, troubleshooting	Can debug Pending, CrashLoopBackOff, ImagePullBackOff, 503s, DNS, networking, and resource issues.
CI/CD	GitHub Actions, self-hosted runners, Jenkins, CloudBees, build/test/scan/deploy gates	Understands secure pipelines, OIDC, artifact promotion, rollback, approvals, and runner hardening.
GitOps / Argo CD	App, ApplicationSet, app-of-apps, sync, health, drift, Helm integration, rollback	Can explain pull-based delivery, self-healing, promotion, and debugging OutOfSync or degraded apps.
Helm	Chart.yaml, values, templates, lint, upgrade, rollback, hooks, environment overrides	Can package reusable applications and debug template/rendering/dependency failures.
Data / MLOps	Databricks, Glue, Redshift, S3 data lake, jobs, secrets, monitoring	Can support data pipelines, workspace automation, data platform security, and job reliability.
SRE / Observability	SLI/SLO, error budget, Prometheus, CloudWatch, logs, traces, incident response	Can troubleshoot production using metrics, logs, traces, dashboards, and postmortems.
Security	IAM, network isolation, secret management, image scanning, RBAC, OIDC, policy as code	Can reduce blast radius, remove long-lived credentials, secure workloads, and pass audits.

Memory trick / clue: For UsefulBI-style cloud/data roles, lead with AWS + Terraform + EKS + CI/CD + Databricks/MLOps + troubleshooting. That combination sounds aligned to cloud, DevOps, and data platform work.

2. Core Workflows to Remember

Production CI/CD to EKS Workflow

Developer PR -> CI tests -> security scan -> build image -> push image to registry -> update Helm values or GitOps repo -> Argo CD sync -> Kubernetes rollout -> health checks -> monitor -> rollback if needed.

Memory trick / clue: Think of it as a pipeline: change -> validation -> approval -> deployment -> verification -> rollback/prevention.

Terraform Safe Change Workflow

Create branch -> edit module -> terraform fmt/validate -> terraform plan -> peer review plan -> approval -> apply with locked remote state -> verify cloud resource -> tag/change record -> monitor.

Memory trick / clue: Think of it as a pipeline: change -> validation -> approval -> deployment -> verification -> rollback/prevention.

Kubernetes Troubleshooting Workflow

Check app symptoms -> kubectl get pods/events -> describe failing object -> logs and previous logs -> resource requests/limits -> service endpoints -> ingress/load balancer -> DNS/network policy -> node health.

Memory trick / clue: Think of it as a pipeline: change -> validation -> approval -> deployment -> verification -> rollback/prevention.

Incident Response Workflow

Detect alert -> declare severity -> assign roles -> gather facts -> mitigate first -> communicate status -> find root cause -> permanent fix -> postmortem -> action items.

Memory trick / clue: Think of it as a pipeline: change -> validation -> approval -> deployment -> verification -> rollback/prevention.

GitOps Promotion Workflow

Dev values -> test -> merge to stage folder -> Argo stage sync -> validation -> PR to prod folder -> approval -> Argo prod sync -> smoke test -> monitor SLO.

Memory trick / clue: Think of it as a pipeline: change -> validation -> approval -> deployment -> verification -> rollback/prevention.

Quick Architecture Diagram - Secure EKS Delivery

```
Developer PR
|
v
GitHub Actions / CloudBees CI
|-- unit tests, lint, SAST, image scan
|-- OIDC assume AWS deploy role
v
Container Registry + Helm/GitOps Repo
|
v
Argo CD Controller in EKS
|
v
Kubernetes Deployment -> Service -> Ingress/ALB -> Users
|
+--> CloudWatch/Prometheus/Splunk logs, metrics, alerts
```

Quick Architecture Diagram - UsefulBI Data Platform

```
Data Sources
|
v
S3 Data Lake ---- Glue Crawlers / ETL ---- Redshift / Databricks
| | |
| v v
| IAM + KMS + Secrets BI / ML / Analytics
|
+--> CloudWatch logs, job metrics, data quality checks, alerts
```

3. AWS Cloud Questions and Answers

Memory trick / clue: UsefulBI interviews can combine classic AWS architecture with data services. Always include security, observability, cost, and failure handling in answers.

Q1. Design a highly available AWS architecture for a web application.

Answer: Use multiple Availability Zones, public subnets for ALB, private subnets for app/EKS nodes, private subnets for databases, NAT gateways for outbound access, autoscaling groups or Karpenter, IAM least privilege, KMS encryption, WAF if public, CloudWatch/Prometheus monitoring, and automated deployment through CI/CD or GitOps. The key tradeoff is cost vs availability, especially NAT gateways, multi-AZ databases, and redundancy.

Memory trick / clue: Public edge, private compute, private data. Remember: ALB outside, pods inside, database deepest.

Q2. Explain VPC, subnet, route table, IGW, NAT Gateway, security group, and NACL.

Answer: A VPC is an isolated network boundary. Subnets divide the VPC by AZ and purpose. Route tables decide where traffic goes. An Internet Gateway lets public subnet resources reach the internet. A NAT Gateway lets private subnet resources initiate outbound internet traffic. Security Groups are stateful instance/ENI firewalls. NACLs are stateless subnet firewalls.

Memory trick / clue: VPC is the neighborhood; subnets are streets; route tables are GPS; SG is house door; NACL is street gate.

Q3. How do you design private EKS nodes with internet access?

Answer: Place worker nodes in private subnets with routes to NAT gateways. The EKS control plane endpoint can be public restricted by CIDR or private-only. Pods that need AWS APIs should use IRSA instead of node credentials. Images should come from ECR through NAT or VPC endpoints. Use VPC endpoints for S3, ECR, CloudWatch, STS, and Secrets Manager to reduce NAT cost and improve security.

Memory trick / clue: Private nodes do not mean no outbound. They initiate outbound through NAT or VPC endpoints.

Q4. What is IRSA and why is it important?

Answer: IAM Roles for Service Accounts maps a Kubernetes service account to an AWS IAM role using OIDC. It lets a pod access AWS services such as S3 or Secrets Manager without storing AWS keys and without overusing the node instance role. It improves least privilege and auditability.

Memory trick / clue: IRSA = pod identity for AWS. One pod, one role, minimal permissions.

Q5. ALB vs NLB: when do you use each?

Answer: ALB is layer 7 HTTP/HTTPS and supports host/path routing, TLS termination, WAF integration, and application-aware routing. NLB is layer 4 TCP/UDP/TLS, very high performance, static IP support, and good for non-HTTP protocols or when preserving source IP matters. For Kubernetes web apps, ALB Ingress is common; for raw TCP services, NLB is common.

Memory trick / clue: ALB understands URLs; NLB understands connections.

Q6. How would you troubleshoot an EC2 instance that cannot reach the internet?

Answer: Check subnet route table, IGW for public subnet or NAT for private subnet, security group egress, NACL outbound/inbound ephemeral ports, instance public IP if public subnet, DNS resolution, OS firewall, and whether a VPC endpoint route is overriding traffic. Use reachability analyzer, VPC flow logs, ping/curl/traceroute where allowed.

Memory trick / clue: Route -> firewall -> address -> DNS -> OS. Always layer the debugging.

Q7. How do you secure S3?

Answer: Block public access by default, use bucket policies with least privilege, enable encryption with SSE-S3 or SSE-KMS, use versioning and lifecycle policies, restrict access through IAM roles and VPC endpoints, enable access logs or CloudTrail data events for sensitive buckets, and use object lock for immutability when required.

Memory trick / clue: S3 safety: Block, Encrypt, Version, Log, Limit.

Q8. How do you reduce AWS cost in a cloud platform?

Answer: Right-size instances, use autoscaling/Karpenter, spot where safe, savings plans/reserved instances for stable workloads, S3 lifecycle tiering, reduce NAT traffic using VPC endpoints, clean unused EBS/EIP/snapshots, set budgets and tags, optimize logs retention, and review expensive services such as NAT, data transfer, Redshift, and Databricks clusters.

Memory trick / clue: Cost leaks: idle compute, NAT data, logs, snapshots, overprovisioned databases.

Q9. How does CloudWatch fit in a DevOps platform?

Answer: CloudWatch collects metrics, logs, alarms, events, and dashboards for AWS resources. It can trigger alarms to SNS, Lambda, PagerDuty, or incident channels. For EKS, it often complements Prometheus/Grafana and Fluent Bit/Splunk/OpenTelemetry pipelines.

Memory trick / clue: CloudWatch is AWS-native eyes and ears.

Q10. How would you use Lambda in DevOps automation?

Answer: Lambda can automate event-driven tasks such as tagging enforcement, snapshot cleanup, alert enrichment, Slack/Webex notifications, S3 file processing, IAM compliance checks, or triggering pipeline/webhook actions. Keep it idempotent, observable, least-privileged, and timeout-aware.

Memory trick / clue: Lambda is glue for events; do not use it for long-running heavy jobs.

4. Terraform / IaC Questions and Answers

Q1. Explain Terraform init, plan, apply, and destroy.

Answer: init downloads providers/modules and configures backend. plan compares configuration with state and real infrastructure to show proposed changes. apply executes approved changes and updates state. destroy removes resources defined in state. In production, plan should be reviewed and apply should be gated.

Memory trick / clue: init prepares, plan predicts, apply performs, destroy deletes.

Q2. What is Terraform state and why is it critical?

Answer: State maps Terraform resources to real cloud resource IDs and stores dependency metadata. Without accurate state, Terraform cannot safely know what exists. Production state should be remote, locked, encrypted, backed up, and access-controlled.

Memory trick / clue: State is Terraform memory. If memory is wrong, actions are dangerous.

Q3. How do you structure Terraform for dev, stage, and prod?

Answer: Use reusable modules for common components and separate environment root modules or folders for dev/stage/prod. Each environment should have separate backend state, variables, accounts or workspaces depending on standards, and separate approval policies. Avoid one giant state file for everything.

Memory trick / clue: Module = blueprint; env folder = actual house.

Q4. What is drift and how do you handle it?

Answer: Drift is when real infrastructure differs from Terraform state/config due to manual changes or external automation. Detect using plan in CI or scheduled drift detection. Fix by importing changes into code, reverting manual changes, or updating state carefully. Avoid fixing by blindly applying without understanding the diff.

Memory trick / clue: Drift means reality and code disagree.

Q5. How do you prevent Terraform from deleting critical resources?

Answer: Use lifecycle prevent_destroy, separate state for critical resources, restricted IAM permissions, manual approval gates, backups/snapshots, deletion protection on services, and code review. For databases, enable deletion protection and final snapshots.

Memory trick / clue: Critical resource safety needs both Terraform guardrails and cloud-native guardrails.

Q6. count vs for_each?

Answer: count creates resources by numeric index and is good for identical resources. for_each creates resources by stable keys and is safer when items are added/removed because it avoids index shifting. For maps or named resources, prefer for_each.

Memory trick / clue: count is list index; for_each is named identity.

Q7. How do you manage secrets in Terraform?

Answer: Do not hardcode secrets. Use AWS Secrets Manager, SSM Parameter Store, Vault, or CI secret stores. Mark variables sensitive, restrict state access because secrets can still end up in state, and prefer passing references/ARNs instead of secret values.

Memory trick / clue: Sensitive hides display, not necessarily state. Protect the backend.

Q8. How do you run Terraform safely in CI/CD?

Answer: Run fmt, validate, security scans, plan on PR, store plan artifacts, require approval, apply only from protected branches, use remote locked state, assume cloud roles via OIDC, separate permissions per environment, and send notifications. Never run production apply from a developer laptop casually.

Memory trick / clue: PR for plan, approval for apply.

5. Kubernetes / EKS Questions and Answers

Q1. Explain Kubernetes architecture.

Answer: The API server is the front door. etcd stores cluster state. Scheduler places pods on nodes. Controller manager reconciles desired state. Kubelet runs pods on each node. Kube-proxy or CNI handles service networking. In EKS, AWS manages the control plane while customers manage node groups, add-ons, policies, and workloads.

Memory trick / clue: API server asks, etcd remembers, scheduler places, controllers reconcile, kubelet runs.

Q2. What happens when you run kubectl apply?

Answer: kubectl sends desired YAML to the API server. The API server validates and stores it in etcd. Controllers notice the desired state and create/update lower-level objects. The scheduler assigns pods to nodes. Kubelet pulls images and starts containers. Status flows back to the API server.

Memory trick / clue: Apply sets desired state; controllers make reality match.

Q3. Pod stuck Pending: how do you debug?

Answer: Run kubectl describe pod and check events. Common causes: insufficient CPU/memory, no matching node selector/taint toleration, PVC not bound, image pull secret issue, quota, pod security policy/admission denial, or cluster autoscaler/Karpenter not provisioning. Check nodes, capacity, storage class, and autoscaler logs.

Memory trick / clue: Pending means not scheduled or not ready to start. Events tell the reason.

Q4. CrashLoopBackOff: how do you debug?

Answer: Check current and previous logs, describe pod events, container exit code, command/args, environment variables, config maps/secrets, health probes, resource limits/OOMKilled, dependency connectivity, and recent deployment changes. If liveness probe is too aggressive, it can cause restarts.

Memory trick / clue: CrashLoop means container starts then dies repeatedly.

Q5. ImagePullBackOff: how do you debug?

Answer: Check image name/tag, registry availability, imagePullSecrets, ECR permissions, node network/NAT/VPC endpoints, repository policy, and whether the image exists. For private ECR, verify node/pod permissions and docker registry auth.

Memory trick / clue: ImagePullBackOff is usually name, auth, network, or missing image.

Q6. Readiness vs liveness vs startup probes.

Answer: Readiness decides whether a pod receives traffic. Liveness decides whether Kubernetes restarts the container. Startup gives slow apps time to boot before liveness starts. Misconfigured probes can cause outages or restart loops.

Memory trick / clue: Readiness = traffic; liveness = restart; startup = patience.

Q7. How do requests and limits affect scheduling?

Answer: Requests reserve CPU/memory and are used by the scheduler to place pods. Limits cap usage. If memory exceeds limit, container can be OOMKilled. CPU limit throttles. Bad requests lead to poor packing or pending pods; bad limits cause throttling or restarts.

Memory trick / clue: Requests schedule; limits restrict.

Q8. How do you expose an app in EKS?

Answer: Create Deployment, Service, and Ingress. For AWS, use AWS Load Balancer Controller to create ALB for HTTP/HTTPS ingress or NLB for TCP/UDP services. Configure TLS certs through ACM/cert-manager, DNS through Route 53/ExternalDNS, and security groups appropriately.

Memory trick / clue: Deployment runs pods, Service finds pods, Ingress exposes HTTP.

Q9. How do you upgrade EKS safely?

Answer: Check version skew, upgrade control plane first, test add-ons, upgrade managed node groups gradually, drain nodes safely with PodDisruptionBudgets, validate workloads, monitor metrics, and have rollback/recreate strategy for nodes. Test in dev/stage before prod.

Memory trick / clue: Control plane first, nodes second, add-ons carefully.

Q10. How do you secure Kubernetes workloads?

Answer: Use namespaces, RBAC least privilege, network policies, non-root containers, read-only root filesystem, image scanning/signing, secrets management, pod security standards, resource quotas, IRSA for AWS access, admission policies such as Kyverno/OPA, and audit logs.

Memory trick / clue: Secure identity, image, runtime, network, and secrets.

6. Helm Deployment Questions and Answers

Q1. What problem does Helm solve?

Answer: Helm packages Kubernetes YAML into reusable charts with templates and values. It allows repeatable deployments, environment-specific overrides, dependency management, release tracking, upgrades, and rollbacks.

Memory trick / clue: Helm is a package manager plus templating for Kubernetes.

Q2. Explain Chart.yaml, values.yaml, templates, and release.

Answer: Chart.yaml stores chart metadata. values.yaml stores default configurable inputs. templates contain Kubernetes manifests with Go templating. A release is an installed instance of a chart with chosen values.

Memory trick / clue: Chart is recipe, values are ingredients, release is cooked meal.

Q3. How do you override values per environment?

Answer: Use separate values files such as values-dev.yaml, values-stage.yaml, and values-prod.yaml; use --set for small overrides; or use Argo CD/Helm valueFiles. Keep secrets outside plain values files unless encrypted with tools like SOPS or External Secrets.

Memory trick / clue: Same chart, different values.

Q4. How do you debug Helm templates?

Answer: Use helm lint, helm template, helm install --dry-run --debug, and inspect rendered YAML. Then validate Kubernetes API errors, missing required values, indentation, invalid selectors, immutable field changes, and failed hooks.

Memory trick / clue: Render first, then apply. Most Helm bugs are rendered YAML bugs.

Q5. How do you rollback a Helm deployment?

Answer: Use helm history to find revisions and helm rollback release revision. Verify pods and services after rollback. In GitOps, rollback should usually be done by reverting the Git commit so desired state matches the rollback.

Memory trick / clue: Helm rollback changes cluster; GitOps rollback changes Git.

7. CI/CD, GitHub Actions, Runners, Jenkins, and CloudBees

Q1. Design a production CI/CD pipeline for EKS.

Answer: Pipeline stages: checkout, lint, unit tests, build image, SAST/dependency scan, container scan, push immutable image tag, generate SBOM, update Helm/GitOps values, deploy to dev, integration tests, approval, promote to stage/prod, smoke test, monitor, and rollback. Use OIDC to assume AWS roles instead of static keys.

Memory trick / clue: Build once, promote the artifact. Do not rebuild differently for prod.

Q2. GitHub-hosted vs self-hosted runners.

Answer: GitHub-hosted runners are managed, ephemeral, easy, and isolated but limited in network customization. Self-hosted runners give private network access and custom tools but require patching, scaling, isolation, and security hardening. For production AWS access, ephemeral self-hosted runners are safer than long-lived shared runners.

Memory trick / clue: Hosted = convenience; self-hosted = control + responsibility.

Q3. How do you secure self-hosted runners?

Answer: Use ephemeral runners, least-privilege IAM, isolated subnets, no shared secrets on disk, restrict repo access, patch runner images, avoid running untrusted fork PRs, use OIDC, monitor logs, and destroy runners after each job.

Memory trick / clue: A runner is a temporary employee with credentials. Do not let it live forever.

Q4. What is GitHub Actions OIDC to AWS?

Answer: GitHub issues a short-lived OIDC token to the workflow. AWS IAM trusts GitHub as an identity provider and allows specific repo/branch/workflow claims to assume a role. This avoids long-lived AWS access keys in GitHub secrets.

Memory trick / clue: OIDC replaces stored keys with short-lived trust.

Q5. Jenkinsfile declarative vs scripted pipeline.

Answer: Declarative pipeline has structured syntax with pipeline, stages, steps, post, and environment blocks. Scripted pipeline is Groovy-based and more flexible but harder to govern. Enterprises often prefer declarative for consistency and readability.

Memory trick / clue: Declarative = guardrails; scripted = flexibility.

Q6. What does CloudBees add over Jenkins?

Answer: CloudBees provides enterprise Jenkins features such as centralized management, governance, controller/agent management, security, compliance, shared libraries, templates, scalability, and support. It is useful in large organizations with many teams and pipelines.

Memory trick / clue: CloudBees is enterprise Jenkins with governance.

Q7. How do approval gates work?

Answer: Approval gates pause promotion to sensitive environments until an authorized person or policy approves. They can include change ticket validation, security scan thresholds, test results, and manual business approval. Gates reduce risk but should not become manual bottlenecks for every low-risk change.

Memory trick / clue: Gate risky changes; automate safe checks.

8. Argo CD and GitOps Questions and Answers

Q1. What is GitOps?

Answer: GitOps uses Git as the source of truth for infrastructure and application desired state. A controller such as Argo CD continuously compares Git with the cluster and reconciles drift. Changes are made through pull requests, giving auditability and rollback through Git history.

Memory trick / clue: Git is desired state; controller makes cluster match.

Q2. Push-based CI/CD vs pull-based GitOps.

Answer: Push-based pipelines deploy directly to the cluster using credentials in CI. Pull-based GitOps has an in-cluster controller pull desired state from Git and apply it. Pull-based reduces external cluster credentials, improves auditability, and detects drift continuously.

Memory trick / clue: Push says CI touches cluster. Pull says cluster watches Git.

Q3. What is an Argo CD Application?

Answer: An Application defines a source repo/path/chart, destination cluster/namespace, project, sync policy, and options. It tells Argo CD what desired state to deploy and where to deploy it.

Memory trick / clue: Application = source + destination + sync rules.

Q4. Sync, health, and drift in Argo CD.

Answer: Sync status shows whether live cluster state matches Git. Health shows whether resources are operational. Drift is the difference between desired Git state and live cluster state. An app can be Synced but Unhealthy if the manifest applied successfully but pods are failing.

Memory trick / clue: Synced means applied; healthy means working.

Q5. How do you debug OutOfSync?

Answer: Compare live vs desired manifests in Argo CD, check ignored differences, generated Helm values, CRD defaulting, admission controller mutations, manual kubectl changes, and resource hooks. Fix the source of truth or configure ignoreDifferences carefully.

Memory trick / clue: OutOfSync means Git and cluster disagree.

Q6. How do you debug Degraded health?

Answer: Check resource health details, kubectl get/describe, pod logs, events, rollout status, service endpoints, ingress, secrets/config, and image versions. Degraded is usually runtime failure, not Git diff failure.

Memory trick / clue: Degraded means applied but not healthy.

Q7. What is app-of-apps?

Answer: A parent Argo CD application deploys child Application manifests. It helps bootstrap many apps, environments, or platform components from one entry point. Use it carefully with clear ownership to avoid confusing cascading changes.

Memory trick / clue: App-of-apps = one app manages many apps.

Q8. What is ApplicationSet?

Answer: ApplicationSet generates many Argo CD Applications from templates using generators such as list, cluster, Git directory, matrix, or pull request. It is useful for multi-cluster and multi-environment deployments.

Memory trick / clue: ApplicationSet is app factory.

Q9. How do you rollback with Argo CD?

Answer: Revert the Git commit or change the image/tag/values back to a known good version, then let Argo sync. Emergency manual rollback is possible but should be reconciled back into Git immediately.

Memory trick / clue: In GitOps, rollback = Git revert.

9. Docker and Container Questions and Answers

Q1. Dockerfile CMD vs ENTRYPOINT.

Answer: ENTRYPOINT defines the main executable for the container. CMD provides default arguments or default command. CMD can be overridden easily; ENTRYPOINT makes the container behave like a specific binary. Many production images use ENTRYPOINT plus CMD defaults.

Memory trick / clue: ENTRYPOINT is the program; CMD is default args.

Q2. What is a multi-stage Docker build?

Answer: A multi-stage build uses one stage to compile/build and a smaller final stage to run. This reduces image size and attack surface by excluding compilers, build tools, and source files from the runtime image.

Memory trick / clue: Build big, run small.

Q3. How do you reduce image size?

Answer: Use slim/distroless base images, multi-stage builds, remove package cache, copy only needed artifacts, pin dependencies, avoid unnecessary layers, and use .dockerignore.

Memory trick / clue: Small images deploy faster and expose less attack surface.

Q4. Why avoid running containers as root?

Answer: Root containers increase blast radius if compromised. Use non-root users, drop capabilities, read-only root filesystem, and Pod Security Standards. Some workloads need specific privileges, but they should be justified and isolated.

Memory trick / clue: Container root is not harmless; reduce privilege.

Q5. How do you debug a container that exits immediately?

Answer: Check docker logs or kubectl logs --previous, inspect command/args, environment variables, mounted files, permissions, missing dependencies, entrypoint script errors, and exit code. Override command with sleep/sh if possible for debugging.

Memory trick / clue: Exits immediately = process ended or crashed. Logs and exit code first.

10. Databricks, Data Platform, and MLOps Questions and Answers

Q1. How would you automate Databricks workspace deployment?

Answer: Use Terraform Databricks provider or cloud-specific modules to create workspace, clusters, jobs, permissions, secrets, instance profiles, and repos. Use CI/CD to deploy notebooks/jobs, enforce cluster policies, and separate dev/stage/prod workspaces or folders.

Memory trick / clue: Treat Databricks like infrastructure plus code, not click-ops.

Q2. How do you manage Databricks secrets?

Answer: Use Databricks secret scopes backed by cloud secret managers where possible, restrict ACLs, avoid printing secrets in notebooks, rotate credentials, and prefer instance profiles/IAM roles for S3 access instead of static keys.

Memory trick / clue: Do not put secrets in notebooks or job parameters.

Q3. How do you optimize Databricks cost?

Answer: Use job clusters instead of always-on all-purpose clusters, autoscaling, spot instances where safe, cluster policies, auto-termination, right-size workers, optimize Spark partitions, cache carefully, and monitor expensive jobs.

Memory trick / clue: Cost killer: idle clusters.

Q4. How do Glue, S3, Redshift, and Databricks fit together?

Answer: S3 stores raw/processed data as a data lake. Glue catalogs metadata and can run ETL. Databricks processes large-scale Spark workloads and ML workflows. Redshift serves analytics/warehouse queries. IAM/KMS secure access and CloudWatch monitors jobs.

Memory trick / clue: S3 lake, Glue catalog/ETL, Databricks compute, Redshift warehouse.

Q5. How do you troubleshoot a slow Spark job?

Answer: Check data skew, partition count, shuffle size, executor memory/CPU, spills, wide transformations, file sizes, cache usage, joins, cluster size, and Spark UI stages/tasks. Optimize with repartitioning, broadcast joins, predicate pushdown, file compaction, and correct cluster sizing.

Memory trick / clue: Spark slowness is often skew, shuffle, or bad partitioning.

11. SRE, Observability, and Troubleshooting Questions and Answers

Q1. What are SLI, SLO, and SLA?

Answer: SLI is a measured reliability indicator such as latency, availability, or error rate. SLO is the internal target for that SLI, such as 99.9% availability. SLA is a contractual promise to customers with consequences. Error budget is the allowed unreliability under the SLO.

Memory trick / clue: SLI measures, SLO targets, SLA promises.

Q2. How do you design observability for EKS?

Answer: Collect metrics with Prometheus/CloudWatch, logs with Fluent Bit/OpenTelemetry/Splunk, traces with OpenTelemetry, and events from Kubernetes. Build dashboards around golden signals: latency, traffic, errors, saturation. Alert on symptoms and SLO burn rate, not every low-level metric.

Memory trick / clue: Metrics say what, logs say why, traces say where.

Q3. What is a good incident response process?

Answer: Triage severity, assign incident commander, communicate status, mitigate quickly, preserve evidence, identify root cause, implement permanent fix, write blameless postmortem, and track action items. During incidents, restoring service comes before perfect root cause.

Memory trick / clue: Mitigate first, analyze second, prevent third.

Q4. How do you handle noisy alerts?

Answer: Map alerts to user impact, remove duplicate alerts, tune thresholds, use burn-rate alerting, add runbooks, route by ownership, and review alert quality after incidents. Every page should require human action.

Memory trick / clue: If no one acts, it should not page.

Q5. How do you do capacity planning?

Answer: Measure current utilization, growth rate, peak patterns, resource saturation, and scaling limits. Load test critical paths, set requests/limits, autoscaling rules, node capacity, database/storage thresholds, and budget forecasts.

Memory trick / clue: Capacity planning = current usage + growth + headroom + failure margin.

12. Security, IAM, Secrets, and Compliance Questions and Answers

Q1. How do you design least privilege IAM?

Answer: Start with required actions and resources, scope permissions to exact ARNs where possible, use conditions, separate roles by workload/environment, avoid wildcards, review access logs, and automate policy checks. Prefer short-lived credentials through role assumption/OIDC.

Memory trick / clue: Allow only who, what, where, and when.

Q2. How do you manage secrets in Kubernetes?

Answer: Avoid plain Kubernetes secrets as the only control. Use External Secrets Operator with AWS Secrets Manager/Vault, encrypt secrets at rest, restrict RBAC, mount secrets only to required pods, rotate secrets, and avoid logging them.

Memory trick / clue: Kubernetes Secret is base64, not a vault.

Q3. How do you secure CI/CD supply chain?

Answer: Use branch protection, signed commits/tags where possible, pinned action versions, dependency scanning, SAST, container scanning, SBOMs, artifact signing, OIDC to cloud, least-privilege runners, and protected deployment environments.

Memory trick / clue: Secure source, build, artifact, deploy.

Q4. What is policy as code?

Answer: Policy as code defines security/compliance rules in code and enforces them automatically through tools such as OPA Gatekeeper, Kyverno, Checkov, tfsec, or Sentinel. Examples: no public S3 buckets, containers cannot run as root, required tags, and approved regions only.

Memory trick / clue: Policy as code turns standards into automated gates.

13. System Design Architecture - DevOps / Cloud / Data Platform

Use these as interview-ready architecture answers. Always explain requirements, components, data/control flow, security, scaling, failure handling, observability, and tradeoffs.

Design a secure EKS platform for multiple application teams.

Use separate AWS accounts or namespaces per environment/team depending on isolation needs. Create VPC with public ALB subnets and private node subnets. Use managed node groups or Karpenter, IRSA, External Secrets, AWS Load Balancer Controller, cert-manager/ACM, Argo CD, Prometheus/Grafana/CloudWatch, centralized logs, RBAC, network policies, and admission policies. Use Terraform for cluster and add-ons, GitOps for workloads. Tradeoffs: shared cluster efficiency vs blast radius; managed add-ons simplicity vs customization.

Memory trick / clue: System design answer order: Requirements -> Architecture -> Security -> Scaling -> Failure -> Observability -> Tradeoffs.

Design CI/CD + GitOps for microservices.

CI builds and tests images, scans, pushes immutable tags, updates GitOps repo values, then Argo CD syncs to the cluster. Environments use separate folders or branches. Promotions happen by PR. Rollback is Git revert. Add approvals for prod, smoke tests after sync, and SLO monitoring. Keep CI from directly mutating prod cluster.

Memory trick / clue: System design answer order: Requirements -> Architecture -> Security -> Scaling -> Failure -> Observability -> Tradeoffs.

Design a data lake and analytics platform.

Use S3 zones: raw, curated, analytics. Glue catalog/crawlers manage metadata. Glue/Databricks jobs transform data. Redshift serves warehouse queries. IAM roles, Lake Formation or bucket policies govern access. KMS encrypts data. CloudWatch/Spark logs monitor jobs. Add data quality checks, lifecycle policies, partitioning, and cost controls.

Memory trick / clue: System design answer order: Requirements -> Architecture -> Security -> Scaling -> Failure -> Observability -> Tradeoffs.

Design a multi-account AWS platform.

Use AWS Organizations with separate accounts for shared networking, dev, stage, prod, security/logging. Centralize CloudTrail, GuardDuty, Security Hub, Config, and logs. Use Transit Gateway or VPC peering/private links. Use IAM Identity Center and cross-account roles. Terraform/Terragrunt manages account baselines. Tradeoff: stronger isolation vs operational complexity.

Memory trick / clue: System design answer order: Requirements -> Architecture -> Security -> Scaling -> Failure -> Observability -> Tradeoffs.

Design disaster recovery for a cloud application.

Define RTO/RPO first. Use multi-AZ for high availability and cross-region backups/replication for disaster recovery. Store images and IaC in durable registries/repos. Back up databases, test restores, use Route 53 failover if needed, and document runbooks. DR must be tested, not assumed.

Memory trick / clue: System design answer order: Requirements -> Architecture -> Security -> Scaling -> Failure -> Observability -> Tradeoffs.

Architecture Diagram - Multi-Account EKS and Data Platform

```
AWS Organization
|
+-- Shared Network Account: VPC, TGW, Route53, shared endpoints
|
+-- Security/Logging Account: CloudTrail, GuardDuty, Config, SIEM export
|
+-- Dev Account: EKS dev, S3 dev, Databricks dev
|
+-- Stage Account: EKS stage, S3 stage, Databricks stage
|
+-- Prod Account: EKS prod, private nodes, ALB/NLB, S3, Glue, Redshift
|
+-- Argo CD deploys apps from Git
+-- CloudWatch/Prometheus/Splunk observe workloads
+-- IAM/IRSA/Secrets Manager secure access
```

14. Scenario-Based Troubleshooting Playbook

Scenario: Argo CD is Healthy but users get 503.

Workflow answer: Check Ingress/ALB target health, Service selector, Endpoints, pod readiness, application port, security groups, network policies, app logs, and recent config changes. Argo health may only mean Kubernetes resources exist, not that external traffic works.

Scenario: Terraform wants to recreate production database.

Workflow answer: Stop. Do not apply. Inspect diff, provider changes, immutable fields, state drift, module refactor, resource address changes, lifecycle settings, and imports/moved blocks. Take backup/snapshot, add `prevent_destroy`, and fix state/code safely.

Scenario: GitHub Actions cannot assume AWS role.

Workflow answer: Check OIDC provider, IAM trust policy conditions for repo/branch/environment, role ARN, permissions, workflow token permissions, branch protection/environment approval, AWS STS availability, and whether repo name/org changed.

Scenario: Pods are pending even though cluster has nodes.

Workflow answer: Check events for insufficient resources, taints/tolerations, node selectors, affinity, PVC binding, quota, PodSecurity/admission, architecture mismatch, and max pods per node/IP exhaustion.

Scenario: AWS bill suddenly increased.

Workflow answer: Check Cost Explorer by service/account/tag, NAT gateway data processing, EC2/EBS idle resources, load balancer hours, CloudWatch logs ingestion/retention, data transfer, Redshift/Databricks clusters, snapshots, and recent deployments.

Scenario: Helm upgrade failed halfway.

Workflow answer: Check helm status/history, Kubernetes events, hooks, immutable field errors, failed jobs, values changes, and rendered YAML. Roll back with helm rollback or Git revert if managed by Argo CD. Fix chart and redeploy.

15. Cheat Sheets and Memory Tricks

Topic	Remember this
AWS VPC	Public subnet = route to IGW. Private subnet = route to NAT or endpoints. Security group is stateful; NACL is stateless.
EKS	Control plane managed by AWS; nodes/add-ons/workloads are your responsibility. IRSA gives pods AWS permissions.
Kubernetes debug	get -> describe -> logs -> events -> endpoints -> ingress -> node -> network.
Terraform	State is critical. Plan before apply. Remote backend, locking, review, approval.
Helm	Lint and template before install. Same chart, different values. Rollback with Helm or Git revert in GitOps.
Argo CD	Synced means desired equals live. Healthy means resources are working. OutOfSync is drift; Degraded is runtime health.
GitHub Actions	Use OIDC, not static AWS keys. Secure runners like production servers.
CloudBees/Jenkins	Enterprise CI needs controller/agent governance, shared libraries, templates, credentials, and approvals.
Observability	Metrics = what, logs = why, traces = where, events = what changed.
Security	Least privilege, short-lived credentials, encrypted secrets, scanned images, policy gates.
System design	Requirements -> components -> flow -> security -> scaling -> failure -> observability -> tradeoffs.

One-Page Memory Map

Cloud platform interview answer map:

1. Network: VPC, subnets, routes, SG/NACL, DNS, endpoints
2. Identity: IAM, OIDC, IRSA, RBAC, least privilege
3. Compute: EC2, EKS, nodes, pods, autoscaling
4. Delivery: GitHub Actions / CloudBees -> image -> Helm -> Argo CD
5. Data: S3 -> Glue/Databricks -> Redshift
6. Reliability: probes, HPA/Karpenter, multi-AZ, backups, rollback
7. Observability: metrics, logs, traces, alerts, SLOs
8. Security: secrets, scanning, policies, encryption, audit
9. Cost: right-size, autoscale, lifecycle, endpoints, retention

16. Behavioral / HR Questions and Answers

Q1. Tell me about yourself.

Answer: Give a 60-90 second summary: current role, strongest DevOps/cloud skills, one relevant project, impact, and why the role matches your direction. Keep it technical but readable.

Q2. Why UsefulBI?

Answer: UsefulBI works around cloud, DevOps, MLOps, data platforms, and automation. A strong answer connects your AWS/Terraform/EKS/CI/CD background to building reliable cloud and data platforms for customers.

Q3. Describe a production incident you handled.

Answer: Use STAR: situation, task, action, result. Include detection, mitigation, communication, root cause, and prevention. Avoid blaming people.

Q4. Tell me about a conflict with developers.

Answer: Explain how you aligned on facts, logs, ownership, and customer impact. Show collaboration: shared runbook, better pipeline checks, clearer readiness probes, or improved alerts.

17. Final Interview Checklist

Before interview	Action
Resume projects	Prepare 2-minute architecture explanation and 3 deep technical details for each project.
AWS	Practice VPC, IAM, EKS, ALB/NLB, S3, Glue, Redshift, Lambda, CloudWatch, cost.
Terraform	Practice state, modules, drift, count vs for_each, import, safe apply.
Kubernetes	Practice Pending, CrashLoopBackOff, ImagePullBackOff, 503, DNS, probes, autoscaling.
CI/CD	Practice GitHub Actions OIDC, runners, Jenkins/CloudBees, approvals, rollback.
GitOps	Practice Argo CD app, sync, health, drift, app-of-apps, ApplicationSet.
System design	Draw architecture first, then explain security, scaling, failures, observability, tradeoffs.
Scenarios	Answer with workflow, not random guesses.

Memory trick / clue: The best interview answers sound like real operations: what you check first, how you prove it, how you fix it, and how you prevent it next time.